Lab *03 – Bash Scripting*

Author: Mateus Guzman Fernandez
Course Section: 003
Date: 11/10/2021

INTRODUCTION

The purpose of this lab is to introduce scripting in Bash using a shell, which is an interface used to communicate with your computer's system and programs to perform tasks. We are going to be learning how to use a variety of concepts in Bash, like variables, assignments, if-else statements, and more, to apply them in scripts. Bash has two modes of operation: Interactive mode and non-interactive mode, with the latter involving scripts.

PROCESS

Breakpoint 1: Variables and Assignments

This step was all about getting used to nano, our editor, and how to assign values to variables. Below is a screenshot of the first script I've ever written in all its glory, which gives out some basic date and time information and a typical greeting at some university in Texas. Now, the reason that my machine was able to determine this to be a Bash script was because of the shebang up top **"#!/bin/bash".** Without that, it would just be ASCII text in a file.

Anyway, here's how my variables work: **user** works by adding the **whoami** command, which just prints the current user's username inside the $'s parentheses. **"echo"** just displays the message in " " out to the system. **today** uses the **date** command to display the date, but the **%D** modulus makes it appear in a mm/dd/yy format. **nowtime** also displays the date, but the **%H, %M, and %S** moduli also add the current hour, minute, and second upon running the script.

```
#!/bin/bash

user=$(whoami)
echo "Howdy and gig em! I am $user!"

today=$(date + "%D")
echo "Today's date is $today"

nowtime=$(date + "%H-%M-%S")
echo "The current time in San Antonio is $nowtime"
```

You can't run a script without adding execute permissions on the file, and to do that, I used the command **chmod +x about-me.sh** to update the file perms, which allowed me to run my script. I followed that up with **"s -al ./about-me.sh** to check if my file existed, then I just ran my script with **./about-me.sh** .

```
mgfubuntu@MateusGuzmanFernandez-VirtualBox:~$ chmod +x about-me.sh
mgfubuntu@MateusGuzmanFernandez-VirtualBox:~$ ls -al ./about-me.sh
-rwxrwxr-x 1 mgfubuntu mgfubuntu 199 Nov 11 23:03 ./about-me.sh
mgfubuntu@MateusGuzmanFernandez-VirtualBox:~$ ./about-me.sh
Howdy and gig em! I am mgfubuntu
Today's date is 11/11/21
The current time in San Antonio is 23:04:27 PM
mgfubuntu@MateusGuzmanFernandez-VirtualBox:~$
```

What is the function of the **$** sign within the script?

The $ sign denotes where the variable we assigned a value to is expressed. For example, "today" is a condensed version of date + "%D", and it makes for an easier way to use that expression throughout the script by just typing a word, in this case.

Breakpoint 2: Filenames and Executables

We started out here by copying the contents of the **about-me.sh** file into a new one called **about-me-v2** using the **cp c**ommand**.** I, then, went and confirmed that the type of file was constant between **about-me.sh** and **about-me-v2.**

Earlier I mentioned that without the shebang up top, the text in the file would just be, well, text. That's exactly what I did; by removing the shebang from the text in the v2 file, the text is not recognized by the system as a Bash script. For this to be able to run, we have to manually call a bash interpreter. The **Bash** interpreter lets this run as a Bash script even without the shebang.

```
mgfubuntu@MateusGuzmanFernandez-VirtualBox:~$ file about-me.sh
about-me.sh: ASCII text
mgfubuntu@MateusGuzmanFernandez-VirtualBox:~$ cp about-me.sh about-me-v2
mgfubuntu@MateusGuzmanFernandez-VirtualBox:~$ file about-me-v2
about-me-v2: ASCII text
mgfubuntu@MateusGuzmanFernandez-VirtualBox:~$ about-me-v2
about-me-v2: command not found
mgfubuntu@MateusGuzmanFernandez-VirtualBox:~$ nano about-me-v2
mgfubuntu@MateusGuzmanFernandez-VirtualBox:~$ bash about-me-v2
Howdy and gig em! I am mgfubuntu
Today's date is 11/12/21
The current time in San Antonio is 14:44:49 PM
```

```
user=$(whoami)
echo "Howdy and gig em! I am $user"

today=$(date +"%D")
echo "Today's date is $today"

nowtime=$(date +"%H:%M:%S %p")
echo "The current time in San Antonio is $nowtime"
```

What does the term executable mean when referring to a script?

The term "executable" for scripts means that your script's code's outputs can be read and are readily available to run.

Breakpoint 3: Appending to a Bash Shell Script

This part tackles appending things to the end of your scripts. The way to do that is through the **“>>”** indicator. Below, I used **“>>”** to append an **echo** command at the end of my script that displayed the Bash version the user was currently running on. I, then, used **cat** to display the contents of **about-me.sh**, and at the end of the script, lo and behold: my **echo** command displaying the Bash version. The screenshot is just a little longer on this one since I didn’t like that the Bash version echo was not spaced apart from the previous line. ***Details are everything, folks.***

```
mgfubuntu@MateusGuzmanFernandez-VirtualBox:~$ echo "The current bash version is $BASH_VERSION" >> about-me.sh
mgfubuntu@MateusGuzmanFernandez-VirtualBox:~$ cat about-me.sh

#!/bin/bash

user=$(whoami)
echo "Howdy and gig em! I am $user"

today=$(date +"%D")
echo "Today's date is $today"

nowtime=$(date +"%H:%M:%S %p")
echo "The current time in San Antonio is $nowtime"
The current bash version is 5.0.17(1)-release
mgfubuntu@MateusGuzmanFernandez-VirtualBox:~$ nano about-me.sh
mgfubuntu@MateusGuzmanFernandez-VirtualBox:~$ cat about-me.sh

#!/bin/bash

user=$(whoami)
echo "Howdy and gig em! I am $user"

today=$(date +"%D")
echo "Today's date is $today"

nowtime=$(date +"%H:%M:%S %p")
echo "The current time in San Antonio is $nowtime"

The current bash version is 5.0.17(1)-release
mgfubuntu@MateusGuzmanFernandez-VirtualBox:~$
```

What is in the script for the Bash shell version? Is it the variable, or the value? What happens when you execute the script again? The **$BASH_VERSION** variable is in the script. When I went to execute my **v2** file to test it, it gave me a syntax error as shown below.

```
mgfubuntu@MateusGuzmanFernandez-VirtualBox:~$ ./about-me-v2
Howdy and gig em! I am mgfubuntu
Today's date is 11/12/21
The current time in San Antonio is 17:28:37 PM
./about-me-v2: line 13: syntax error near unexpected token `('
./about-me-v2: line 13: `my bash shell version is 5.0.17(1)-release'
```

Breakpoint 4: Functions

This point has us use functions to give some more information about the user's system. We use the function **user_details** to echo a variety of other sets of info like username in **$user**, home directory in **$HOME,** and more. I, then, executed the script with the function included, and it reflects the changes made in the function.

```
#!/bin/bash

#username and date
user=$(whoami)
echo "Howdy and gig em! I am $user"

today=$(date +"%D")
echo "Today's date is $today"

nowtime=$(date +"%H:%M:%S %p")
echo "The current time in San Antonio is $nowtime"

#user's system information
function user_details {
        echo "Howdy and gig em! I am $user"
        echo "Home Directory: $HOME"
        echo "ID: $(id)"
        echo "Your Bash shell version is $BASH_VERSION"
        lslogins -u
}

#calling the user-details function
user_details
```

```
mgfubuntu@MateusGuzmanFernandez-VirtualBox:~$ nano about-me.sh
mgfubuntu@MateusGuzmanFernandez-VirtualBox:~$ ./about-me.sh
Howdy and gig em! I am mgfubuntu
Today's date is 11/12/21
The current time in San Antonio is 15:08:39 PM
Howdy and gig em! I am mgfubuntu
Home Directory: /home/mgfubuntu
ID: uid=1000(mgfubuntu) gid=1000(mgfubuntu) groups=1000(mgfubuntu),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),120(lpadmin),132(lxd),133(sambashare)
Your Bash shell version is 5.0.17(1)-release
 UID USER      PROC PWD-LOCK PWD-DENY LAST-LOGIN GECOS
   0 root        99                          root
1000 mgfubuntu   79                       14:39 Mateus Guzman Fernandez,,,
```

When was your last login? What is your UID and GID?

My last login was at 14:39 or 2:39 PM today, my UID is 1000 and my GID is also 1000.

Breakpoint 5: Conditional Statements

In this point, I employed the use of functions a bit more, this time spicing it up with a conditional statement built in it. We used the function **os_type** to give us more info on the user's operating system upon the execution of the script. The conditional statements used in it were put there to determine which OS the user is running on, and uses a couple of commands I honestly could not explain to you how they work. The **type -t** commands are the lifeblood of the statements, and that is what is used to determine whether your OS is **MSWIN, macOS, or Linux**.

```
#print OS type
function os_type {
        if type -t wevtutil &> /dev/null
        then
            OS=MSWIN
        elif type -t scutil &> /dev/null
        then
            OS=macOS
        else
            OS=Linux
        fi
        echo "My operating system is $OS"
}

#calling the os-type function
os_type
```

```
mgfubuntu@MateusGuzmanFernandez-VirtualBox:~$ ./about-me.sh
Howdy and gig em! I am mgfubuntu
Today's date is 11/12/21
The current time in San Antonio is 15:18:13 PM
Howdy and gig em! I am mgfubuntu
Home Directory: /home/mgfubuntu
ID: uid=1000(mgfubuntu) gid=1000(mgfubuntu) groups=1000(mgfubuntu),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),120(lpadmin),132(lxd),133(sambashare)
Your Bash shell version is 5.0.17(1)-release
 UID USER      PROC PWD-LOCK PWD-DENY LAST-LOGIN GECOS
   0 root       100                          root
1000 mgfubuntu   79                       14:39 Mateus Guzman Fernandez,,,
My operating system is Linux
```

What happens if you include a function body, but do not call a function within a script? Do you receive a warning or an error?

If you include a function body without calling a function within a script, you receive an error.

Breakpoint 6: Loops

I dived into creating loops in this part, and we once again led with a function here, this time **list_scripts** was its name. I used **list_scripts** to house a for-loop which listed any file whose name included **".sh"** and printed them at the end of my **about-me.sh** script. In my second shot, I executed the script and it brought me just one name, that being **about-me.sh.**

```
#print shell scripts in current directory
function list_scripts {
        for SCRIPTNAME in $(ls | grep ".sh")
        do
            echo $SCRIPTNAME
        done
}

#calls list_scripts
list_scripts
```

```
mgfubuntu@MateusGuzmanFernandez-VirtualBox:~$ ./about-me.sh
Howdy and gig em! I am mgfubuntu
Today's date is 11/12/21
The current time in San Antonio is 15:24:43 PM
Howdy and gig em! I am mgfubuntu
Home Directory: /home/mgfubuntu
ID: uid=1000(mgfubuntu) gid=1000(mgfubuntu) groups=1000(mgfubuntu),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),120(lpadmin),132(lxd),133(sambashare)
Your Bash shell version is 5.0.17(1)-release
 UID USER      PROC PWD-LOCK PWD-DENY LAST-LOGIN GECOS
   0 root        99                               root
1000 mgfubuntu   77                       14:39 Mateus Guzman Fernandez,,,
My operating system is Linux
about-me.sh
```

What is the purpose of the variable **`SCRIPTNAME`** in the given code?

$SCRIPTNAME is a variable meant to substitute for the stored value given to it.

Breakpoint 7: Positional Parameters and Associative Arrays

Our last part here goes into positional parameters and associative arrays. The purpose is to get the total hours spent and number of days spent working on this lab, and we did this by declaring the **lab_time** array. In the output shot below, the total number of hours that I put as an example was 7 and the days were 6. I had to execute my function and put those two numbers after it for the number of hours and days to appear, if I did not do this, they would appear blank.

```
#gather input for lab times and store info in an associative array
declare -A lab_time
lab_time[total_hrs_spent]=$1
lab_time[num_days_spent]=$2

#print associative array
function lab_log
        for key in "${lab_time[@]}"
        do
           echo "$key: ${lab_time[$key]}"
        done
}

#calls lab_log
lab_log
```

```
mgfubuntu@MateusGuzmanFernandez-VirtualBox:~$ ./about-me.sh 7 6
Howdy and gig em! I am mgfubuntu
Today's date is 11/12/21
The current time in San Antonio is 15:39:19 PM
Howdy and gig em! I am mgfubuntu
Home Directory: /home/mgfubuntu
ID: uid=1000(mgfubuntu) gid=1000(mgfubuntu) groups=1000(mgfubuntu),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),120(lpadmin),132(lxd),133(sambashare)
Your Bash shell version is 5.0.17(1)-release
 UID USER      PROC PWD-LOCK PWD-DENY LAST-LOGIN GECOS
   0 root        99                               root
1000 mgfubuntu   77                        14:39 Mateus Guzman Fernandez,,,
My operating system is Linux
about-me.sh
total_hrs_spent: 7
num_days_spent: 6
```

What is the purpose/function of the [@] symbol in the given code?

The purpose of the @ symbol is to reference all the values stored in the array as an individual variable and not lump them all together into one.

CONCLUSION

Please provide a conclusion based on these prompts:

The goals of the techniques and tools used are to provide a basic understanding of the Bash language, helping us achieve a polyglot status in programming, as well. The basic coding we did in this lab personally helped me break my poor disposition to polyglot coding and learning anything outside of Python. I realized that Bash was not too far from what I was already learning in Python and having a chance to learn the basics in another coding language actually helps me understand all of them more. Of course, we can take the basics of Bash and build on our knowledge, eventually reaching higher levels of skill in not

only Bash but other languages, as previously mentioned. Coding is an integral part of becoming a cyber security expert, so having a lab that hard focuses on an important and relevant coding language is a blessing that will surely prove useful in our careers. These techniques and tools are highly effective. They form the basis of the languages and learning these basics is the first step to unlocking the full capabilities of what coding can do. Arrays appear to be the most useful of all these tools and seem to have the most practical uses within coding as its own standalone concept. Organization and efficiency is critical for any company, and that is the main purpose of arrays. I believe this lab is highly relevant for most of the reasons from before: Coding is critical to us and is a foundational concept in cyber security that we will need to learn to progress in our careers, so having a detailed introduction to a coding language is something hugely necessary. I personally did not think we would dive into coding in this class, but I am very glad we got a chance to do so. The lab itself is very easy to follow along and does a great job of explaining the functions of each of the breakpoints and the coding concepts behind them. All in all, learning the basics of Bash is surely going to come in handy not just later on, but promptly, too.

REFERENCES

Internet Resources

The Evolution of Malicious Shell Scripts (Fiser and Oliviera, TrendMicro, September 2020)

How to Use Nano Command Line Text Editor in Ubuntu 18.04 | 16.04 (Richard, Website for Students, July 2019).

How To Format Date For Display or Use In a Shell Script (nixCraft, Gite, August 2021).

Collaboration

I did not collaborate with anyone on this lab.